

1 lintquarto: a package for running linters, formatters, 2 static type checkers and code analysis tools on Python 3 code in Quarto Markdown files

4 **Amy Heather**  ^{1*}

5 ¹ University of Exeter, England * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 25 June 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a ¹⁴
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)) ¹⁵.

6 Summary

7 lintquarto enables researchers to apply Python linters, formatters, static type checkers, and
8 code analysis tools to Python code embedded within Quarto Markdown (.qmd) files. Quarto
9 ([Allaire et al., 2022](#)) has become widely adopted in research for creating websites, papers,
10 reports, and documentation that integrate code, narrative, and outputs. However, standard
11 Python code quality tools cannot natively run on Quarto Markdown files. lintquarto bridges
12 this gap, enabling researchers to maintain Python code quality standards in their Quarto
13 documents.

Statement of need

14 Researchers can embed Python code in Quarto documents using two formats: Jupyter notebooks
15 (.ipynb) or Quarto Markdown. Quarto Markdown is the recommended approach. It integrates
16 better with version control because code and text are stored as plain text, and outputs are
17 generated separately during rendering, not stored in the source file. In contrast, .ipynb stores
18 outputs and metadata in JSON, making diffs harder to read.

19 Despite these advantages, Python researchers using Quarto Markdown face a key limitation:
20 code quality tools cannot natively run on Quarto Markdown files. Code quality tools include:

- 21 ▪ Linters: Identify style violations and potential bugs.
- 22 ▪ Formatters: Automatically format code to address style issues.
- 23 ▪ Static type checkers: Verify type annotations.
- 24 ▪ Code analysis tools: Detect dead code or measure complexity.

25 These tools are essential for maintaining high code standards and ensuring research code is
26 understandable, maintainable, and reliable.

28 State of the field

29 For Jupyter notebooks, nbqa is the established solution, wrapping Python linters and formatters
30 so they can run on .ipynb files ([nbQA contributors, 2025](#)).

31 For R, the lintr package supports code quality checking across Quarto Markdown, R Markdown
32 and plain R files ([Hester et al., 2025](#)).

33 However, no equivalent Python tool exists for researchers using Quarto Markdown files.

Software design

The command-line interface for `lintquarto` accepts three primary arguments: the code quality tools to run, the paths to include, and paths to exclude.

The core workflow converts each Quarto Markdown file into a temporary Python file. Where relevant, line numbers are preserved so that error messages map directly back to the original Quarto source. The converter handles Quarto-specific syntax including Quarto chunk options, include directives, in-line annotations (e.g., `#<<`, `# <1>`) and false positives triggered by chunk boundaries.

After conversion, `lintquarto` runs the selected code quality tool on the temporary Python file, captures its output, and rewrites reported paths to reference the original Quarto document. For formatters, `lintquarto` can apply changes back to the source `.qmd` file.

This approach enables seamless integration, with no modifications to the underlying existing code quality tools, and makes it easy to extend to incorporate new tools. As of publication, the package supports all major Python code quality tools:

- `flake8` ([Python Code Quality Authority, 2025a](#))
- `pycodestyle` ([Python Code Quality Authority, 2025b](#))
- `pydoclint` ([pydoclint contributors, 2025](#))
- `pyflakes` ([Python Code Quality Authority, 2025c](#))
- `pylint` ([Pylint contributors, 2026](#))
- `ruff` ([Astral, 2026](#))
- `mypy` ([Python, 2026](#))
- `pyrefly` ([Facebook, 2026](#))
- `pyright` ([Microsoft, 2026](#))
- `basedpyright` ([basedpyright contributors, 2026](#))
- `pytype` ([Google, 2025](#))
- `vulture` ([vulture contributors, 2026](#))
- `radon` ([Radon contributors, 2024](#))

`lintquarto` is distributed via PyPI and conda-forge, and has minimal dependencies. The software has undergone external peer review by PyOpenSci ([issue 257 in their software review repository](#)), providing additional assurance regarding code quality, documentation, and maintainability.

Research impact statement

By enabling Python code quality checking in Quarto Markdown, `lintquarto` supports reproducible and reliable scientific research. Early community engagement has included external users submitting feature requests and bug reports, demonstrating clear demand for improved tooling in this space. The project is also recognised in the *Awesome Quarto* repository ([Canouil, 2026](#)).

`lintquarto` is already being used in a range of projects. One example from the author's work is the DES RAP Book, a step-by-step guide for building discrete-event simulation models in Python and R as part of a reproducible analytical pipeline, where it is used to maintain the quality of code within the Quarto book website ([Heather et al., 2026](#)).

AI usage disclosure

This project was written and maintained by hand, with occasional use of Perplexity AI during development (specific models and versions varied over time).

78 AI assistance was used for small, targeted tasks in the code (e.g. help when troubleshooting,
79 identifying issues, refining docstrings, improving code structure) and paper (e.g., suggesting
80 improvements for grammar or clarity), rather than to generate complete functions or substantial
81 content.

82 All code and design decisions were reviewed and finalised by a human.

83 Acknowledgements

84 The author gratefully acknowledges all contributors to `lintquarto`, including those who
85 contributed code, documentation, bug reports, feature requests, testing, and feedback. These
86 contributions have helped to shape and improve the package and its documentation.

87 This work was supported by the Medical Research Council under grant number
88 [MR/Z503915/1].

89 References

- 90 Allaire, J. J., Teague, C., Xie, Y., & Dervieux, C. (2022). *Quarto*. Zenodo. <https://doi.org/10.5281/zenodo.5960048>
- 91
- 92 Astral. (2026). *Ruff*. <https://github.com/astal-sh/ruff>
- 93 basedpyright contributors. (2026). *Basedpyright*. [https://github.com/DetachHead/](https://github.com/DetachHead/basedpyright)
94 [basedpyright](https://github.com/DetachHead/basedpyright)
- 95 Canouil, M. (2026). *Awesome Quarto*. <https://github.com/mcanouil/awesome-quarto>
- 96 Facebook. (2026). *Pyrefly*. Meta. <https://github.com/facebook/pyrefly>
- 97 Google. (2025). *Pytype*. Google. <https://github.com/google/pytype>
- 98 Heather, A., Monks, T., Mustafee, N., Harper, A., & Alidoost, F. (2026). *DES RAP*
99 *Book: Reproducible Discrete-Event Simulation in Python and R*. [https://github.com/](https://github.com/pythonhealthdatascience/des_rap_book)
100 [pythonhealthdatascience/des_rap_book](https://github.com/pythonhealthdatascience/des_rap_book)
- 101 Hester, J., Angly, F., Chirico, M., Hyde, R., Kun, R., Patil, I., & Rosenstock, A. (2025).
102 Static Code Analysis for R. *Journal of Open Source Software*, 10(108), 7240. <https://doi.org/10.21105/joss.07240>
103
- 104 Microsoft. (2026). *Pyright*. Microsoft. <https://github.com/microsoft/pyright>
- 105 nbQA contributors. (2025). *nbQA*. nbQA-dev. <https://github.com/nbQA-dev/nbQA>
- 106 pydoclint contributors. (2025). *Pydoclint*. <https://github.com/jsh9/pydoclint>
- 107 Pylint contributors. (2026). *Pylint*. <https://github.com/pylint-dev/pylint>
- 108 Python. (2026). *Mypy*. Python. <https://github.com/python/mypy>
- 109 Python Code Quality Authority. (2025a). *flake8*. <https://github.com/PyCQA/flake8>
- 110 Python Code Quality Authority. (2025b). *Pycodestyle*. [https://github.com/PyCQA/](https://github.com/PyCQA/pycodestyle)
111 [pycodestyle](https://github.com/PyCQA/pycodestyle)
- 112 Python Code Quality Authority. (2025c). *Pyflakes*. <https://github.com/PyCQA/pyflakes>
- 113 Radon contributors. (2024). *Radon*. <https://github.com/rubik/radon>
- 114 vulture contributors. (2026). *Vulture*. <https://github.com/jendrikseipp/vulture>